Cloud-native AI

Wie diese Veranstaltung funktioniert

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27
Bachelor Angewandte Informatik · Vertiefung Software Engineering



Über mich

- **40 Jahre**, verheiratet, zwei Kinder
- Professor für Softwaresysteme und KI
- **Büro:** O205
- **Mail:** pascal.laube@htwg-konstanz.de
- **Sprechstunde:** jederzeit, kurze Mail vorher genügt


Automating
Next
Generation
Mobility





1 · Worum es geht

Vom Prototyp in den Betrieb

Ein Sprachmodell im Notebook zum Laufen zu bringen ist leicht.

Spannend wird es danach.

Was danach kommt

Sobald echte Nutzer euer AI-System verwenden, muss es mehr können als eine Antwort ausgeben:

- **Wissen abrufen**, das das Modell nicht gelernt hat
- **Werkzeuge nutzen** und mehrere Schritte planen
- **gemessen werden:** Ist es gut, und bleibt es gut, wenn ich etwas ändere?
- **containerisiert, ausgeliefert und skaliert** werden
- **überwacht** und **sicher ausgerollt** werden

> Genau diesen Weg gehen wir im Semester. Agenten sind dabei ein Baustein unter mehreren. Der Schwerpunkt liegt auf Engineering und Betrieb.

Was heißt „cloud-native“?

Cloud-native heißt: Anwendungen so bauen, dass sie in dynamischen Umgebungen zuverlässig laufen. **Container**, **deklarative Konfiguration**, **automatische Skalierung**, **Observability**.

AI-Workloads stellen diese Prinzipien auf die Probe:

Klassischer Webservice	AI-System
startet in Sekunden	Modell mit vielen GB muss erst geladen werden
läuft auf jeder CPU	braucht oft eine GPU
gleiche Eingabe, gleiche Ausgabe	Ausgaben variieren, Qualität ist schwer zu messen
Kosten pro Anfrage gering	Kosten pro Token spürbar

Die CNCF nennt das **Cloud Native Artificial Intelligence**, kurz CNAI. Daher auch der Kurzname dieses Moduls.

> **Was ist die CNCF?** Die *Cloud Native Computing Foundation* ist eine gemeinnützige Stiftung unter dem Dach der Linux Foundation, gegründet 2015. Sie betreut zentrale Open-Source-Projekte wie **Kubernetes** und **Prometheus** und prägt, was „cloud-native“ heißt.

Was ihr am Ende gebaut habt

Ein eigenes AI-System, **Schicht für Schicht**:

- Modell als austauschbarer Baustein
- Wissenszugriff: File-based oder RAG
- Agent mit Werkzeugen und MCP
- Evaluation, die bei jedem Push läuft
- Betrieb im Kubernetes-Cluster mit eigenem Model Serving
- Gateway, Guardrails, Monitoring, sicherer Rollout



2 · Die 14 Termine

Dienstags, 09:45 Vorlesung · 11:30 Übung · O/201

Teil A: Vom Modell zum Agenten

#	Datum	Thema
01	06.10.	Was ist Cloud-native AI? Kursvorstellung
02	13.10.	Foundation Models verstehen
03	20.10.	Prompt Engineering
04	27.10.	Wissen abrufen: File-based Retrieval und RAG
05	03.11.	Agenten und Werkzeuge, MCP
06	10.11.	Orchestrierung, Memory, Multi-Agent
07	17.11.	Tag des Lernens: Hackathon oder Checkpoint

Teil B: Messen und Betreiben

#	Datum	Thema
08	24.11.	Evaluation
09	01.12.	Kubernetes für AI-Workloads
10	08.12.	Model Serving und Inferenzoptimierung
11	15.12.	Produktionsarchitektur: Gateway, Guardrails, Caching
12	22.12.	Monitoring, Drift, Continual Learning
13	12.01.	Sicher ausrollen, verantwortlich betreiben
14	19.01.	Abschluss und Klausurvorbereitung

Vom 24.12. bis 06.01. ist Weihnachtspause.

3 · Wie die Veranstaltung läuft

So läuft ein Dienstag

Wann	Was passiert
vorher	Skriptkapitel lesen, Videos schauen, **Quiz** in Moodle machen
09:45–11:15	Vorlesung: Konzepte, Live-Demo, Diskussion. In der Mitte: Anwesenheitsliste
11:30–13:00	Übung: Ihr baut die Schicht der Woche in euer System

> Jede Vorlesung beginnt mit derselben Architekturfolie: **Hier stehen wir, diese Schicht kommt heute dazu.**

Die Skripte basieren auf Fachbüchern

Damit der rote Faden stimmt, folgt jedes Skriptkapitel einem Buchkapitel:

Buch	Wofür
Chip Huyen: *AI Engineering* (2025)	Hauptlinie: Modelle, Prompting, RAG, Evaluation, Inferenz, Architektur
Huß, Zonca: *Generative AI on Kubernetes* (2026)	Betrieb: Serving, GPUs, Kubernetes, Observability. **Frei als PDF**
Michael Albada: *Building Applications with AI Agents* (2025)	Agenten, Orchestrierung, Security
Chip Huyen: *Designing ML Systems* (2022)	Drift, Monitoring, Test in Production

Ihr müsst die Bücher nicht kaufen. Das Skript ist eigenständig lesbar und enthält alles, was ihr für Übung und Klausur braucht.

> Wer die Bücher haben möchte, schreibt mich bitte direkt an: **pascal.laube@htwg-konstanz.de**

4 · Das Praktikum

Ein System, sieben Abgaben

Sieben Abgaben im Zwei-Wochen-Takt

Abgabe	nach VL	Was euer System danach kann
P1	02	läuft im Dev-Container, ruft ein Modell lokal und per API auf, liefert strukturierte Ausgaben
P2	04	versionierte Prompts, Wissenszugriff (File-based oder RAG) in Docker Compose
P3	06	Agent mit Werkzeugen, eigener MCP-Server, Orchestrierung
P4	08	Eval-Suite mit LLM-as-Judge, läuft in CI
P5	10	Betrieb im Kubernetes-Cluster mit eigenem Model Serving
P6	12	Gateway, Guardrails, Tracing und Metriken
P7	14	Canary-Rollout und Security-Check

Der Funktionsumfang jeder Abgabe ist vorgegeben. Die genauen Aufgaben und Fristen stehen jeweils in Moodle.

Jede Abgabe hat zwei Teile

Kern
Pflicht für alle.

Die Schicht der Woche, lauffähig und nachvollziehbar dokumentiert.

Zusatz
Entfällt, wenn ihr am Semesterende **mindestens 80 % Anwesenheit** nachweist.

Eine zusätzliche Aufgabe, die das Thema vertieft.

Harte Kriterien für jede Abgabe:

- Das System startet reproduzierbar mit einem dokumentierten Befehl
- Keine Secrets im Repository, Konfiguration über Umgebungsvariablen
- Abhängigkeiten sind gepinnt

Einzeln oder im Team?

Das hängt davon ab, **wie viele ihr seid**. Das klären wir heute gemeinsam.

> Egal wie die Antwort ausfällt: Jede Person muss erklären können, was im eigenen System passiert und warum.

5 · Anwesenheit

Wer da ist, hat weniger Praktikum.

Ab 80 % Anwesenheit entfallen die Zusatzteile der Abgaben.

So funktioniert die Regel

- In jeder Vorlesung erfasse ich in der Mitte die **Anwesenheit**: QR-Code am Beamer scannen, fertig
- **Ausgewertet wird am Semesterende**
- Habt ihr **mindestens 80 %** der Vorlesungen besucht, entfallen die **Zusatzteile** aller Abgaben
- Liegt ihr darunter, müssen **alle Zusatzteile abgegeben worden sein**
- Zusatzteile sind **nicht nachreichbar**

> Das heißt praktisch: Wer nicht sicher ist, ob er oder sie auf 80 % kommt, gibt die Zusatzteile jede Woche mit ab. Wer regelmäßig kommt, spart sich diese Arbeit.

Gezählt wird ab der zweiten Vorlesung (13.10.): 13 Termine, 80 % sind 11. Ihr könnt also zwei Vorlesungen verpassen.

6 · Prüfung

Zwei Leistungen

Leistung	Grundlage	Ergebnis
Praktikum	alle sieben Abgaben bestanden (Kern, und je nach Anwesenheit Zusatz)	bestanden / nicht bestanden
Klausur	60 Minuten, am Ende des Semesters	**die Note**

Für die 6 ECTS braucht ihr beides.

Die Klausur

- **60 Minuten**, Termin im Prüfungszeitraum
- **Konzept- und Transferfragen** zu allen 14 Vorlesungen, szenariobasiert wie die Vorbereitungsquizze
- **Architekturaufgaben:** ein gegebenes System analysieren, die fehlende Schicht benennen, eine Entscheidung begründen

> Die beste Vorbereitung: die Quizze vor jeder Vorlesung, die Kontrollfragen am Ende jedes Skriptkapitels und euer eigenes System. Wer es gebaut hat, versteht die Architekturaufgaben.

Was ihr braucht

Was	Wofür
Laptop mit Docker	jede Übung läuft in Containern
Git-Account (GitHub oder GitLab)	eure Abgaben
Academic Cloud Login (HTWG)	Zugriff auf Sprachmodelle über die GWDG
Grundkenntnisse Python	die meisten Beispiele sind in Python

Für Kubernetes reicht ein lokaler Cluster auf dem Laptop. Das richten wir gemeinsam ein, wenn es so weit ist.

Was von euch erwartet wird

Erwartung	Details
Da sein	spart euch Arbeit im Praktikum
Vorbereitung	Skript, Videos und Quiz vor der Vorlesung
Kontinuität	Abgaben pünktlich, regelmäßige Commits
Eigenständigkeit	KI-Tools sind erlaubt und erwünscht. Verstehen müsst ihr euer System trotzdem selbst

Fragen?

Prof. Dr. Pascal Laube
pascal.laube@htwg-konstanz.de

HTWG Konstanz · Cloud-native AI · WS 2026/27